*(Name of the College)*

**OBJECT-ORIENTED PROGRAMMING (OOP)**
**MINI PROJECT**

# LIBRARY MANAGEMENT SYSTEM

*using C++*

**Submitted By**

**Student Name :** _____

**Roll Number :** _____

**Class :** _____

**Subject :** _____

**College Name :** _____

**Teacher's Name :** _____

# TABLE OF CONTENTS

**1. Introduction** ..... **2**

**2. Objectives** ..... **3**

**3. OOP Concepts Used** ..... **4**

3.1 Class ..... 4
3.2 Object ..... 4
3.3 Encapsulation ..... 4
3.4 Inheritance ..... 5
3.5 Polymorphism ..... 5
3.6 Abstraction ..... 6
3.7 Summary of OOP concepts in the project ..... 6

**4. Features** ..... **7**

**5. Algorithm** ..... **9**

**6. Source Code** ..... **11**

**7. Sample Output** ..... **19**

7.1 Starting the program and displaying books when the library is empty ..... 19
7.2 Adding new books ..... 20
7.3 Input validation while adding (duplicate ID and empty title) ..... 20
7.4 Displaying all books ..... 21
7.5 Searching for a book by ID ..... 21
7.6 Searching for a book by title ..... 22
7.7 Issuing a book ..... 23
7.8 Returning a book ..... 24
7.9 Invalid menu choice and exiting the program ..... 24

**8. Code Explanation** ..... **25**

8.1 Header files ..... 25
8.2 Helper functions ..... 25
8.3 Class LibraryItem (abstract base class) ..... 25
8.4 Class Book (derived class) ..... 26
8.5 Class Library ..... 26
8.6 The main() function and objects ..... 27
8.7 How availability is updated correctly ..... 27
8.8 Common viva questions ..... 27

**9. Conclusion** ..... **29**

# 1. Introduction

A **Library Management System** is a computer program that helps a library keep track of its books. In a traditional library, the librarian writes the details of every book in a register, searches for books by hand and notes down who has borrowed which book. This is slow, and mistakes are very common. A Library Management System does all of this work on the computer, quickly and accurately.

This mini project is a **console-based Library Management System** written in **C++** using **Object-Oriented Programming (OOP)**. The program shows a simple menu. The librarian chooses an option, such as adding a book, searching for a book, issuing a book to a student or taking a book back. The program then performs the task and shows the result on the screen.

In this project, every book is treated as an **object**. Each book object remembers its own ID, title, author, whether it is available or issued, and the name of the student who currently holds it. All the library work is done by a **Library** class that keeps a list of these book objects.

**How the system helps to manage library books**

- **Keeps proper records:** every book is stored with its ID, title and author, so nothing is lost or forgotten.
- **Shows availability instantly:** the system tells whether a book is *Available* or *Issued*, and to whom it is issued.
- **Saves time:** a book can be found at once by its ID or by (part of) its title, instead of searching shelves or registers.
- **Avoids mistakes:** the program does not allow two books with the same ID, and it does not allow a book that is already issued to be issued again.
- **Updates automatically:** when a book is issued or returned, its availability and record are updated immediately.

## 2. Objectives

The main objectives of this project are:

1. **To manage books:** add new books to the library with a unique ID, title and author name.
2. **To display books:** show a neat table of all books along with their current status (Available or Issued).
3. **To search for books:** find a book quickly by its **ID** or by its **title**.
4. **To issue books:** give a book to a student and automatically mark it as *Issued*.
5. **To return books:** accept a returned book and automatically mark it as *Available* again.
6. **To maintain book records:** keep track of availability, the student who holds the book and how many times each book has been issued.
7. **To apply OOP concepts:** use classes, objects, encapsulation, inheritance, polymorphism and abstraction in a real, working program.
8. **To create a user-friendly program:** provide a menu-driven interface with proper input validation so that wrong input does not crash the program.

# 3. OOP Concepts Used

Object-Oriented Programming is a way of writing programs in which we think of the program as a group of **objects** that work together, just like real things in the real world. In a library, the real things are books and the library itself, so they become objects in our program. This section explains each OOP concept in simple words and shows exactly where it is used in the project code.

## 3.1 Class

A **class** is a blueprint (or template) for creating objects. It describes what data an object will store (data members) and what actions it can perform (member functions). A class by itself does not use memory for real data; it is only a design.

**In this project:** three classes are created – `LibraryItem` (a general library item), `Book` (a book in the library) and `Library` (the whole library that manages the books).

```
class Library
{
private:
    vector<Book> books;     // list of all books
public:
    void addBook();         // functions that work on the books
    void displayAllBooks() const;
    // ... other functions ...
};
```

*(Shortened view – the full class is in Section 6.)*

## 3.2 Object

An **object** is a real, usable copy of a class. If a class is a blueprint of a house, an object is the actual house built from it. Many objects can be made from one class, and each object keeps its own data.

**In this project:** `library` is an object of the `Library` class created in `main()`. Every time a new book is added, a new `Book` object is created and stored inside the library.

```
Library library;                        // object of class Library (in main)

books.push_back(Book(id, title, author));   // new Book object (in addBook)
```

## 3.3 Encapsulation

**Encapsulation** means wrapping data and the functions that use that data together inside a class, and **hiding the data** from the outside world using `private`. Other parts of the program can use the data only through public member functions.

**In this project:** the data members of the `Book` class – `author`, `available`, `issuedTo` and `timesIssued` – are **private**. No outside code can change a book's availability directly. It can

be changed only through the public functions `issueBook()` and `returnBook()`, which first check whether the action is allowed. This protects the data from accidental mistakes, for example issuing a book that is already issued.

```
class Book : public LibraryItem
{
private:                  // hidden from outside
    string author;
    bool available;
    string issuedTo;
    int timesIssued;
public:                   // safe way to use the data
    bool issueBook(const string& studentName);
    bool returnBook();
    // ...
};
```

## 3.4 Inheritance

**Inheritance** means creating a new class (child or derived class) from an existing class (parent or base class). The child class automatically gets the data and functions of the parent, so we do not have to write them again. It represents an "is-a" relationship.

**In this project:** `Book` is derived from `LibraryItem`. A book *is a* library item, so it inherits `itemId`, `title`, `getId()` and `getTitle()` from `LibraryItem` and adds its own members such as `author` and `available`. In the future, other items such as magazines or journals could be added by inheriting from the same base class.

```
class LibraryItem { ... };               // parent (base) class

class Book : public LibraryItem { ... }; // child (derived) class
```

## 3.5 Polymorphism

**Polymorphism** means "many forms". The same function call can behave differently depending on the object that receives it. In C++ this is done with **virtual functions** and function **overriding**: the base class declares a function as `virtual`, and the child class provides its own version of it.

**In this project:** `displayDetails()` is declared as a virtual function in `LibraryItem` and is overridden in `Book`. In `displayAllBooks()`, a pointer of the base type `LibraryItem` is used to call `displayDetails()`. At run time, C++ automatically calls the `Book` version of the function. If another class (for example `Magazine`) were added later, the same line of code would call that class's own version.

```
// In LibraryItem (base class)
virtual void displayDetails() const = 0;

// In Book (derived class) - overriding the function
void displayDetails() const override { ... }
```

```
// In Library::displayAllBooks()
const LibraryItem* item = &books[i];
item->displayDetails();      // calls Book's version
```

## 3.6 Abstraction

**Abstraction** means showing only the important features and hiding the unnecessary details. It lets us say *what* an object does without worrying about *how* it does it. In C++, an **abstract class** is a class that has at least one **pure virtual function** (a function written with `= 0`). We cannot create objects of an abstract class; it only acts as a common template for its child classes.

**In this project:** `LibraryItem` is an abstract class because `displayDetails()` is a pure virtual function. It only says that every library item must be able to display its details; the child class `Book` decides how. Abstraction is also seen in `main()`: the user simply calls `library.issueBook()` without needing to know how the books are searched, checked and updated inside the class.

```
virtual void displayDetails() const = 0;   // pure virtual function
```

## 3.7 Summary of OOP concepts in the project

| OOP Concept | Where it is used | Benefit in this project |
|---|---|---|
| Class | `LibraryItem`, `Book`, `Library` | Organises the program into logical parts. |
| Object | `library` in `main()`; each `Book` stored in the vector | Each book keeps its own record. |
| Encapsulation | Private data members of `Book`; public `issueBook()` and `returnBook()` | Availability cannot be changed in a wrong way. |
| Inheritance | `Book` inherits from `LibraryItem` | Reuses ID and title code; easy to extend. |
| Polymorphism | Virtual `displayDetails()` called through a `LibraryItem` pointer | The correct display function runs automatically. |
| Abstraction | Abstract class `LibraryItem`; simple `Library` functions used by `main()` | Hides complex details and defines a common design. |

# 4. Features

The program is menu driven. The following table lists the features and the menu option used to reach each of them.

| Feature | Menu Option | Description |
|---|---|---|
| Add new books | 1 | Adds a new book by taking its ID, title and author. A new book is always *Available*. Duplicate IDs are rejected. |
| Display all available books | 2 | Shows all books in a table with ID, title, author, status, the student it is issued to and how many times it has been issued. Totals of available and issued books are also shown. |
| Search for a book by ID | 3 | Finds the book with the given ID and displays its details. |
| Search for a book by title | 4 | Finds all books whose title contains the entered word or phrase. Capital and small letters are treated the same. |
| Issue a book | 5 | Issues an available book to a student, stores the student's name and changes the status to *Issued*. A book that is already issued cannot be issued again. |
| Return a book | 6 | Takes back an issued book, clears the student's name and changes the status to *Available*. |
| Exit | 7 | Ends the program. |

**Book availability and records**

For every book, the system maintains the following record automatically:

- **Book ID** – a unique positive number.
- **Title** and **Author** – entered when the book is added.
- **Availability status** – *Available* when the book is in the library and *Issued* when a student has taken it.
- **Issued To** – the name of the student who currently holds the book (shown as *None* when the book is available).
- **Times issued** – a counter that increases by one every time the book is issued.

**Additional features**

- **Input validation:** IDs and menu choices must be positive whole numbers; title, author and student name cannot be empty. Wrong input shows an error message and the program asks again.
- **Duplicate protection:** two books cannot have the same ID.
- **Clear messages:** the program always tells the user whether an operation was successful or why it failed.

# 5. Algorithm

The steps below describe how the program works and how the user interacts with it.

**Main algorithm**

1. **Start** the program.
2. Create a `Library` object. At the beginning the library has no books.
3. Display the **main menu** (options 1 to 7).
4. Ask the user to enter a choice. If the input is not a positive whole number, show an error message and ask again.
5. According to the choice entered, do one of the following:
   - **1** – Add a new book.
   - **2** – Display all books.
   - **3** – Search a book by ID.
   - **4** – Search a book by title.
   - **5** – Issue a book.
   - **6** – Return a book.
   - **7** – Show a goodbye message and go to the last step.
   - **Any other number** – Show "Invalid choice" and continue.
6. After the selected task is finished, go back to **Step 3** and display the menu again.
7. **Stop** the program when the user chooses option 7.

**Option 1 – Add a new book**

1. Ask the user to enter the **Book ID**.
2. If a book with the same ID already exists, show an error message and return to the menu.
3. Ask for the **Title** and the **Author** name (they must not be empty).
4. Create a new `Book` object, mark it as *Available*, and store it in the library list.
5. Display "Book added successfully!".

**Option 2 – Display all books**

1. If the library has no books, display "No books in the library yet." and return to the menu.
2. Otherwise print the table heading.
3. Print the details of every book one by one and count how many are available.
4. Print the total number of books, available books and issued books.

**Option 3 – Search a book by ID**

1. Ask the user to enter the **Book ID**.
2. Look through the list of books for that ID.
3. If found, display the details of the book; otherwise display “No book found”.

**Option 4 – Search a book by title**

1. Ask the user to enter a **title** or part of a title.
2. Convert the entered text and each book title to small letters.
3. Display every book whose title contains the entered text.
4. If no book matches, display “No book found”.

**Option 5 – Issue a book**

1. Ask the user to enter the **Book ID**.
2. If the book is not found, display a message and return to the menu.
3. If the book is already issued, display the name of the student who has it and return to the menu.
4. Otherwise ask for the **student name**.
5. Change the status to *Issued*, store the student name and increase the issue count by 1.
6. Display the success message.

**Option 6 – Return a book**

1. Ask the user to enter the **Book ID**.
2. If the book is not found, display a message and return to the menu.
3. If the book is not issued, display “This book was not issued” and return to the menu.
4. Otherwise change the status to *Available* and clear the student name.
5. Display the success message.

**Option 7 – Exit**

1. Display the goodbye message and end the program.

# 6. Source Code

The complete C++ program is given below. It has been compiled and tested with the GNU C++ compiler (`g++`). It uses only basic C++ features and works in any standard C++ compiler such as Code::Blocks, Dev-C++, Visual Studio or an online C++ compiler.

**How to compile and run**

- Save the code in a file named `library.cpp`.
- Compile it using the command: `g++ library.cpp -o library`
- Run it using: `./library` (on Windows: `library.exe`).
- If you use an IDE such as Code::Blocks or Dev-C++, simply paste the code in a new file and press **Compile & Run**.

**Program: library.cpp**

```
// ----------------------------------------------------------------
//  Library Management System
//  A simple OOP mini project in C++
//  Concepts used: Class, Object, Encapsulation, Inheritance,
//                 Abstraction, Polymorphism, Constructors
// ----------------------------------------------------------------
#include <iostream>   // for cin, cout
#include <string>     // for string
#include <vector>     // for vector (stores many books)
#include <iomanip>    // for setw (neat table output)
#include <sstream>    // for stringstream (checking numeric input)
#include <cstdlib>    // for exit()
using namespace std;

// ----------------------------------------------------------------
//  Helper functions for safe input and text handling
// ----------------------------------------------------------------

// Reads a positive whole number. Keeps asking until input is valid.
int readInt(const string& prompt)
{
    string line;
    int value;
    char extra;

    while (true)
    {
        cout << prompt;
        if (!getline(cin, line))            // input ended unexpectedly
        {
            cout << "\nInput ended. Exiting program.\n";
            exit(0);
        }

        stringstream ss(line);
        // Valid only if: a number is read, nothing extra follows, value > 0
        if (ss >> value && !(ss >> extra) && value > 0)
            return value;

        cout << "Invalid input! Please enter a positive whole number.\n";
    }
}
```

```
// Reads a non-empty line of text. Keeps asking until input is valid.
string readText(const string& prompt)
{
    string line;

    while (true)
    {
        cout << prompt;
        if (!getline(cin, line))            // input ended unexpectedly
        {
            cout << "\nInput ended. Exiting program.\n";
            exit(0);
        }

        // Accept only if the line has at least one non-space character
        if (line.find_first_not_of(" \t") != string::npos)
            return line;

        cout << "Invalid input! This field cannot be empty.\n";
    }
}

// Converts a string to lowercase (used for case-insensitive search)
string toLowerCase(string text)
{
    for (size_t i = 0; i < text.length(); i++)
    {
        if (text[i] >= 'A' && text[i] <= 'Z')
            text[i] = text[i] + 32;
    }
    return text;
}

// Prints the heading line of the book table
void printTableHeader()
{
    cout << left
         << setw(5)  << "ID"
         << setw(24) << "Title"
         << setw(18) << "Author"
         << setw(11) << "Status"
         << setw(13) << "Issued To"
         << setw(6)  << "Times" << endl;
    cout << string(77, '-') << endl;
}

// ----------------------------------------------------------------
//  Abstract base class: LibraryItem  (ABSTRACTION)
//  It cannot be used directly because displayDetails() is a
//  pure virtual function. Child classes must define it.
// ----------------------------------------------------------------
class LibraryItem
{
protected:                      // visible to child classes only
    int itemId;
    string title;

public:
    // Constructor: sets the id and title of the item
    LibraryItem(int id, string itemTitle)
    {
        itemId = id;
        title = itemTitle;
    }
```

```
    // Virtual destructor (good practice for base classes)
    virtual ~LibraryItem() {}

    // Getter functions
    int getId() const { return itemId; }
    string getTitle() const { return title; }

    // Pure virtual function (makes this class abstract)
    virtual void displayDetails() const = 0;
};

// ---------------------------------------------------------------
//  Derived class: Book  (INHERITANCE)
//  Book inherits id and title from LibraryItem.
// ---------------------------------------------------------------
class Book : public LibraryItem
{
private:                        // hidden from outside (ENCAPSULATION)
    string author;
    bool available;             // true = in library, false = issued
    string issuedTo;            // name of the student holding the book
    int timesIssued;            // how many times the book was issued

public:
    // Constructor: a new book is always available when added
    Book(int id, string bookTitle, string bookAuthor)
        : LibraryItem(id, bookTitle)
    {
        author = bookAuthor;
        available = true;
        issuedTo = "None";
        timesIssued = 0;
    }

    // Getter functions (read-only access to private data)
    string getAuthor() const { return author; }
    bool isAvailable() const { return available; }
    string getIssuedTo() const { return issuedTo; }

    // Issues the book to a student. Returns false if already issued.
    bool issueBook(const string& studentName)
    {
        if (!available)
            return false;

        available = false;
        issuedTo = studentName;
        timesIssued++;
        return true;
    }

    // Returns the book to the library. Returns false if not issued.
    bool returnBook()
    {
        if (available)
            return false;

        available = true;
        issuedTo = "None";
        return true;
    }

    // Overrides the pure virtual function (POLYMORPHISM)
```

```
    void displayDetails() const override
    {
        cout << left
             << setw(5)  << itemId
             << setw(24) << title
             << setw(18) << author
             << setw(11) << (available ? "Available" : "Issued")
             << setw(13) << issuedTo
             << setw(6)  << timesIssued << endl;
    }
};

// ------------------------------------------------------------------
//  Class: Library
//  Stores all books and performs every library operation.
// ------------------------------------------------------------------
class Library
{
private:
    vector<Book> books;         // list of all books

    // Returns the position of a book with the given id, or -1
    int findIndexById(int id) const
    {
        for (size_t i = 0; i < books.size(); i++)
        {
            if (books[i].getId() == id)
                return (int)i;
        }
        return -1;
    }

public:
    // 1. Add a new book
    void addBook()
    {
        int id = readInt("Enter Book ID: ");

        if (findIndexById(id) != -1)
        {
            cout << "Error: A book with ID " << id << " already exists!\n";
            return;
        }

        string title = readText("Enter Book Title: ");
        string author = readText("Enter Author Name: ");

        books.push_back(Book(id, title, author));   // create object
        cout << "Book added successfully!\n";
    }

    // 2. Display all books
    void displayAllBooks() const
    {
        if (books.empty())
        {
            cout << "No books in the library yet.\n";
            return;
        }

        int availableCount = 0;

        printTableHeader();
        for (size_t i = 0; i < books.size(); i++)
        {
```

```
            // A LibraryItem pointer calls Book's version (POLYMORPHISM)
            const LibraryItem* item = &books[i];
            item->displayDetails();

            if (books[i].isAvailable())
                availableCount++;
        }

        cout << string(77, '-') << endl;
        cout << "Total books: " << books.size()
             << " | Available: " << availableCount
             << " | Issued: " << books.size() - availableCount << endl;
    }

    // 3. Search a book by ID
    void searchById() const
    {
        int id = readInt("Enter Book ID to search: ");
        int index = findIndexById(id);

        if (index == -1)
        {
            cout << "No book found with ID " << id << ".\n";
            return;
        }

        cout << "Book found:\n";
        printTableHeader();
        books[index].displayDetails();
    }

    // 4. Search books by title (full or partial, ignores capital letters)
    void searchByTitle() const
    {
        string keyword = readText("Enter Book Title to search: ");
        string lowerKeyword = toLowerCase(keyword);
        int matches = 0;

        for (size_t i = 0; i < books.size(); i++)
        {
            string lowerTitle = toLowerCase(books[i].getTitle());

            if (lowerTitle.find(lowerKeyword) != string::npos)
            {
                if (matches == 0)
                {
                    cout << "Matching book(s):\n";
                    printTableHeader();
                }
                books[i].displayDetails();
                matches++;
            }
        }

        if (matches == 0)
            cout << "No book found with title containing \""
                 << keyword << "\".\n";
    }

    // 5. Issue a book to a student
    void issueBook()
    {
        int id = readInt("Enter Book ID to issue: ");
        int index = findIndexById(id);
```

```
        if (index == -1)
        {
            cout << "No book found with ID " << id << ".\n";
            return;
        }

        if (!books[index].isAvailable())
        {
            cout << "Sorry, this book is already issued to "
                 << books[index].getIssuedTo() << ".\n";
            return;
        }

        string studentName = readText("Enter Student Name: ");
        books[index].issueBook(studentName);

        cout << "Book \"" << books[index].getTitle()
             << "\" issued to " << studentName << " successfully!\n";
    }

    // 6. Return a book
    void returnBook()
    {
        int id = readInt("Enter Book ID to return: ");
        int index = findIndexById(id);

        if (index == -1)
        {
            cout << "No book found with ID " << id << ".\n";
            return;
        }

        if (books[index].isAvailable())
        {
            cout << "This book was not issued. "
                 << "It is already in the library.\n";
            return;
        }

        // Save the student name before the record is reset
        string studentName = books[index].getIssuedTo();
        books[index].returnBook();

        cout << "Book \"" << books[index].getTitle()
             << "\" returned by " << studentName << " successfully!\n";
    }
};

// Shows the main menu
void showMenu()
{
    cout << "\n===== LIBRARY MANAGEMENT SYSTEM =====\n";
    cout << "1. Add New Book\n";
    cout << "2. Display All Books\n";
    cout << "3. Search Book by ID\n";
    cout << "4. Search Book by Title\n";
    cout << "5. Issue Book\n";
    cout << "6. Return Book\n";
    cout << "7. Exit\n";
    cout << "=====================================\n";
}

// ----------------------------------------------------------------
```

```
//  main(): creates the Library object and runs the menu loop
// ------------------------------------------------------------------
int main()
{
    Library library;            // object of class Library
    int choice;

    do
    {
        showMenu();
        choice = readInt("Enter your choice: ");
        cout << endl;

        switch (choice)
        {
            case 1: library.addBook();        break;
            case 2: library.displayAllBooks(); break;
            case 3: library.searchById();     break;
            case 4: library.searchByTitle();  break;
            case 5: library.issueBook();      break;
            case 6: library.returnBook();     break;
            case 7:
                cout << "Thank you for using the Library Management System. "
                     << "Goodbye!\n";
                break;
            default:
                cout << "Invalid choice! "
                     << "Please enter a number between 1 and 7.\n";
        }
    } while (choice != 7);

    return 0;
}
```

# 7. Sample Output

This section shows a sample run of the program in a console window. The output was produced by running the exact source code given in Section 6. The text typed by the user appears after each prompt (for example, after “Enter your choice: ”).

**Note:** the main menu is displayed after every operation. To save space, it is shown in full only once (in Section 7.1) and is later replaced by the line `[ ...main menu is displayed again... ]`.

## 7.1 Starting the program and displaying books when the library is empty

The program starts with the main menu. Option 2 is chosen before any book is added, so the library is empty.

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add New Book
2. Display All Books
3. Search Book by ID
4. Search Book by Title
5. Issue Book
6. Return Book
7. Exit
=====================================
Enter your choice: 2

No books in the library yet.
```

## 7.2 Adding new books

Three books are added using option 1.

```
[ ...main menu is displayed again... ]
Enter your choice: 1

Enter Book ID: 101
Enter Book Title: Programming in C++
Enter Author Name: Bjarne Stroustrup
Book added successfully!

[ ...main menu is displayed again... ]
Enter your choice: 1

Enter Book ID: 102
Enter Book Title: Data Structures
Enter Author Name: Seymour Lipschutz
Book added successfully!

[ ...main menu is displayed again... ]
Enter your choice: 1

Enter Book ID: 103
Enter Book Title: Database Systems
Enter Author Name: Ramez Elmasri
Book added successfully!
```

## 7.3 Input validation while adding (duplicate ID and empty title)

The program rejects a duplicate Book ID (101 already exists) and asks again when an empty title is entered.

```
[ ...main menu is displayed again... ]
Enter your choice: 1

Enter Book ID: 101
Error: A book with ID 101 already exists!

[ ...main menu is displayed again... ]
Enter your choice: 1

Enter Book ID: 104
Enter Book Title:
Invalid input! This field cannot be empty.
Enter Book Title: Operating Systems
Enter Author Name: Silberschatz
Book added successfully!
```

## 7.4 Displaying all books

Option 2 displays all books. All four books are currently available.

```
[ ...main menu is displayed again... ]
Enter your choice: 2

ID   Title                  Author           Status     Issued To    Times
------------------------------------------------------------------------
101  Programming in C++     Bjarne Stroustrup Available  None         0
102  Data Structures        Seymour Lipschutz Available  None         0
103  Database Systems       Ramez Elmasri     Available  None         0
104  Operating Systems      Silberschatz      Available  None         0
------------------------------------------------------------------------
Total books: 4 | Available: 4 | Issued: 0
```

## 7.5 Searching for a book by ID

Option 3 searches by ID – first for an existing book (102) and then for a book that does not exist (999).

```
[ ...main menu is displayed again... ]
Enter your choice: 3

Enter Book ID to search: 102
Book found:
ID   Title                  Author            Status     Issued To    Times
------------------------------------------------------------------------
102  Data Structures        Seymour Lipschutz Available  None         0

[ ...main menu is displayed again... ]
Enter your choice: 3

Enter Book ID to search: 999
No book found with ID 999.
```

## 7.6 Searching for a book by title

Option 4 searches by title. The word “DATA” matches two books even though it is typed in capital letters; “python” matches nothing.

```
[ ...main menu is displayed again... ]
Enter your choice: 4

Enter Book Title to search: DATA
Matching book(s):
ID   Title                  Author            Status     Issued To    Times
------------------------------------------------------------------------
102  Data Structures        Seymour Lipschutz Available  None         0
103  Database Systems       Ramez Elmasri     Available  None         0

[ ...main menu is displayed again... ]
Enter your choice: 4

Enter Book Title to search: python
No book found with title containing "python".
```

## 7.7 Issuing a book

Option 5 issues book 101 to a student. The second attempt to issue the same book is rejected, and an unknown ID (555) gives an error. The list is displayed afterwards to show that the status, the student name and the totals have been updated.

```
[ ...main menu is displayed again... ]
Enter your choice: 5

Enter Book ID to issue: 101
Enter Student Name: Rahul Sharma
Book "Programming in C++" issued to Rahul Sharma successfully!

[ ...main menu is displayed again... ]
Enter your choice: 5

Enter Book ID to issue: 101
Sorry, this book is already issued to Rahul Sharma.

[ ...main menu is displayed again... ]
Enter your choice: 5

Enter Book ID to issue: 555
No book found with ID 555.

[ ...main menu is displayed again... ]
Enter your choice: 2

ID   Title                   Author            Status     Issued To    Times
------------------------------------------------------------------------------
101  Programming in C++      Bjarne Stroustrup Issued     Rahul Sharma 1
102  Data Structures         Seymour Lipschutz Available  None         0
103  Database Systems        Ramez Elmasri     Available  None         0
104  Operating Systems       Silberschatz      Available  None         0
------------------------------------------------------------------------------
Total books: 4 | Available: 3 | Issued: 1
```

## 7.8 Returning a book

Option 6 returns book 101. A second attempt to return the same book is rejected. The list shows that the book is Available again and its issue count is 1.

```
[ ...main menu is displayed again... ]
Enter your choice: 6

Enter Book ID to return: 101
Book "Programming in C++" returned by Rahul Sharma successfully!

[ ...main menu is displayed again... ]
Enter your choice: 6

Enter Book ID to return: 101
This book was not issued. It is already in the library.

[ ...main menu is displayed again... ]
Enter your choice: 2

ID   Title                   Author            Status     Issued To    Times
------------------------------------------------------------------------------
101  Programming in C++      Bjarne Stroustrup Available  None         1
102  Data Structures         Seymour Lipschutz Available  None         0
103  Database Systems        Ramez Elmasri     Available  None         0
104  Operating Systems       Silberschatz      Available  None         0
------------------------------------------------------------------------------
Total books: 4 | Available: 4 | Issued: 0
```

## 7.9 Invalid menu choice and exiting the program

An invalid menu input (“abc”) and an invalid choice (9) are handled, and then option 7 ends the program.

```
[ ...main menu is displayed again... ]
Enter your choice: abc
Invalid input! Please enter a positive whole number.
Enter your choice: 9

Invalid choice! Please enter a number between 1 and 7.

[ ...main menu is displayed again... ]
Enter your choice: 7

Thank you for using the Library Management System. Goodbye!
```

# 8. Code Explanation

This section explains the important parts of the program in simple language.

## 8.1 Header files

| Header | Purpose |
|---|---|
| `<iostream>` | Gives `cin` and `cout` for input and output. |
| `<string>` | Gives the `string` type to store text such as titles and names. |
| `<vector>` | Gives `vector`, a resizable list that stores all the `Book` objects. |
| `<iomanip>` | Gives `setw()` to print the book table in neat columns. |
| `<sstream>` | Gives `stringstream`, used to check that the user typed a valid number. |
| `<cstdlib>` | Gives `exit()`, used to stop the program safely if the input ends unexpectedly. |

## 8.2 Helper functions

- **`readInt()`** – shows a prompt, reads a whole line and accepts it only if it is a positive whole number with no extra characters. Otherwise it prints an error and asks again. This prevents the program from crashing when the user types letters instead of numbers.
- **`readText()`** – reads a line of text and accepts it only if it contains at least one non-space character, so titles, authors and names cannot be empty.
- **`toLowerCase()`** – converts capital letters to small letters. It is used so that searching by title ignores capital and small letters.
- **`printTableHeader()`** – prints the heading line of the table (ID, Title, Author, Status, Issued To, Times).
- **`showMenu()`** – prints the main menu.

## 8.3 Class LibraryItem (abstract base class)

`LibraryItem` stores the information that is common to every item in the library.

- **Data members:** `itemId` and `title`. They are `protected`, which means they are hidden from the outside but can be used by child classes such as `Book`.
- **Constructor:** `LibraryItem(int id, string itemTitle)` sets the ID and title when an object is created.

- **Member functions:** `getId()` and `getTitle()` return the ID and title. The word `const` at the end means the function does not change the object.
- **Pure virtual function:** `displayDetails() const = 0;` makes the class abstract. Every child class must provide its own version.
- **Virtual destructor:** `virtual ~LibraryItem() {}` is a good-practice line for base classes that have virtual functions.

## 8.4 Class Book (derived class)

`Book` inherits from `LibraryItem` and stores everything that is specific to a book.

| Data member | Type | Meaning |
|---|---|---|
| `itemId` | int | Book ID (inherited from `LibraryItem`). |
| `title` | string | Book title (inherited from `LibraryItem`). |
| `author` | string | Name of the author. |
| `available` | bool | `true` if the book is in the library, `false` if it is issued. |
| `issuedTo` | string | Name of the student holding the book, or "None". |
| `timesIssued` | int | Number of times the book has been issued. |

**Constructor**

The constructor `Book(int id, string bookTitle, string bookAuthor)` runs automatically when a `Book` object is created. It first calls the parent constructor `LibraryItem(id, bookTitle)` and then sets `author`, makes the book available (`available = true`), sets `issuedTo` to "None" and `timesIssued` to 0. So every new book starts as an available book with no issue history.

**Member functions**

- `getAuthor()`, `isAvailable()`, `getIssuedTo()` – getter functions that let other classes read the private data safely.
- `issueBook(studentName)` – if the book is already issued it returns `false`. Otherwise it sets `available = false`, stores the student's name, increases `timesIssued` by 1 and returns `true`.
- `returnBook()` – if the book is not issued it returns `false`. Otherwise it sets `available = true`, resets `issuedTo` to "None" and returns `true`.
- `displayDetails()` – overrides the base-class function and prints one row of the table for this book.

## 8.5 Class Library

The `Library` class manages all the books. Its only data member is `vector<Book> books`, a list that stores all `Book` objects. The class has no user-written constructor; the compiler provides a default one, which starts with an empty list.

- **`findIndexById(id)`** (private) – searches the vector and returns the position of the book with that ID, or -1 if it is not found. It is used by several other functions.
- **`addBook()`** – reads ID, title and author, rejects duplicate IDs, creates a `Book` object and stores it in the vector using `push_back()`.
- **`displayAllBooks()`** – prints every book using a `LibraryItem` pointer (polymorphism) and then prints the totals of available and issued books.
- **`searchById()`** – finds a book using `findIndexById()` and displays it.
- **`searchByTitle()`** – converts the keyword and each title to small letters and uses `find()` to check whether the keyword is part of the title. All matching books are displayed.
- **`issueBook()`** and **`returnBook()`** – ask for the book ID, check that the book exists and is in the correct state, and then call the `Book` functions to change its status.

## 8.6 The main() function and objects

In `main()`, one object of the `Library` class is created: `Library library;`. A `do...while` loop displays the menu, reads the user's choice with `readInt()` and uses a `switch` statement to call the matching function of the `library` object. The loop continues until the user enters 7.

## 8.7 How availability is updated correctly

The program makes sure that availability is always correct by using a single place to change it and by checking the current state first:

- **Issuing:** `Library::issueBook()` first checks `isAvailable()`. If the book is already issued, the program shows who has it and stops. Only when the book is available does it ask for the student name and call `Book::issueBook()`, which sets `available` to `false`, stores the name and increases the issue count.
- **Returning:** `Library::returnBook()` first checks that the book is really issued. Only then does it call `Book::returnBook()`, which sets `available` to `true` and clears the student name. The student name is saved in a variable before clearing so that it can be shown in the success message.
- **Encapsulation** guarantees that no other code can change `available` directly, so the status always matches the real situation.

## 8.8 Common viva questions

| Question | Answer |
|---|---|
| What is a class and what is an object? | A class is a blueprint; an object is a real copy created from that blueprint. Example: `Book` is a class and each book stored in the library is an object. |
| Why are the data members private? | To protect the data (encapsulation). They can be changed only through safe public functions such as `issueBook()` and `returnBook()`. |
| Which class is abstract and why? | `LibraryItem`, because it has a pure virtual function `displayDetails() = 0`. We cannot create its object. |
| Where is inheritance used? | `Book` is derived from `LibraryItem` and inherits `itemId`, `title`, `getId()` and `getTitle()`. |
| Where is polymorphism used? | The virtual function `displayDetails()` is called through a `LibraryItem` pointer in `displayAllBooks()`, and the `Book` version runs. |
| What does the constructor do? | It runs automatically when an object is created and sets the starting values, e.g. a new book is Available with `issuedTo = "None"`. |
| Why is `vector` used? | It stores any number of `Book` objects and grows automatically when new books are added. |
| How is a book prevented from being issued twice? | The program checks `isAvailable()` before issuing; if the book is already issued, it shows an error message. |

# 9. Conclusion

The Library Management System developed in this project is a simple, menu-driven C++ program that manages the main tasks of a small library. It allows the user to add new books, display all books, search for a book by ID or title, issue a book to a student and return it. The availability of every book and its record (who has it and how many times it has been issued) are updated automatically and correctly, and the input validation ensures that wrong input does not crash the program.

**Usefulness:** the system replaces slow and error-prone paper registers with a fast and accurate digital record. It can be used by a school or college library, and it can easily be extended in the future with features such as saving the data in a file, due dates and fines.

**OOP concepts learned:** while developing this project we learned how to design a program using **classes and objects**; how **encapsulation** protects data using private members and public functions; how **inheritance** allows `Book` to reuse the code of `LibraryItem`; how **polymorphism** lets a base-class pointer call the correct overridden function; and how **abstraction** is achieved with an abstract class and simple functions that hide the internal details. We also practised constructors, vectors, string handling and input validation.

Overall, the project shows how OOP makes a program easier to understand, maintain and extend, and it gave us practical experience that will be useful in larger software projects.